

고등부 1. 줄임말

풀이 작성자: 김준원

S 의 길이는 N , T 의 길이는 M 이라고 하자.

부분문제 1 (S 와 T 가 ‘a’만으로 이루어져 있다)

모든 문자가 ‘a’이기 때문에, S 는 ‘a’가 N 개, T^k 는 ‘a’가 kM 개 있는 문자열이다. 언제 S 가 T^k 의 줄임말이 될까? 모든 문자가 ‘a’이므로, T^k 의 길이가 S 보다 길거나 같으면 줄임말이 된다. 이제, $kM \geq N$ 인 최소의 k 를 찾는 문제가 된다. 답은 $\frac{N}{M}$ 을 소숫점 아래에서 올림한 값이 된다. C언어 코드로는 $(N + M - 1) / M$ 과 같이 쓸 수 있다.

부분문제 2 ($N \leq 100, M \leq 100$)

본 문제를 풀기 위해서 조금 더 쉬운 다음 문제를 생각한다. ‘두 문자열 A 와 B 가 주어졌을 때에, B 가 A 의 줄임말인가?’ 아래는 이 문제를 $O(|A| + |B|)$ 에 해결하는 알고리즘이다. ($|A|$ 는 A 의 길이, $|B|$ 는 B 의 길이)

1. A 의 가장 왼쪽에서부터 B 의 첫번째 글자를 찾는다.
2. 위에서 찾은 인덱스 다음부터 B 의 두번째 글자를 찾는다.
3. 위에서 찾은 인덱스 다음부터 B 의 세번째 글자를 찾는다.
4. ...
5. 위 과정을 끝까지 진행할 수 있으면(즉, B 의 마지막 글자까지 모든 글자를 A 에서 찾을 수 있으면) B 는 A 의 줄임말이다. 그렇지 않고 중간에 실패하면, B 는 A 의 줄임말이 아니다.

위 알고리즘은 문자열 A 에서의 위치와, 문자열 B 에서의 위치를 동시에 관리한다는 의미에서 ‘투 포인터’(Two pointers) 알고리즘이라고 부르기도 한다. 이 알고리즘을 이해했다면, 바로 부분문제 2를 풀 수 있게 된다. S 가 T 의 줄임말인지, T^2 의 줄임말인지, T^3 의 줄임말인지, ... 이를 차례대로 줄임말이 될 때까지 판별한다. 줄임말이 되는 것이 불가능하지 않은 이상 답은 언제나 N 이하이고, 따라서 만약 $T^1, \dots, T^N$ 까지 검사한 결과 S 가 어떤 것의 줄임말도 아니라면, -1을 출력한다. 시간 복잡도는 $O(N^2M)$ 이다.

부분문제 3 ($N \leq 10\,000, M \leq 100$)

부분문제 2의 풀이는 동일한 작업을 공연히 N 번 반복한다. 위 알고리즘을 단 한 번만 수행하는 최적화를 수행할 수 있다. 문자열 T 의 처음과 끝을 이어붙이자. 즉, S 가 T 의 줄임말인지 판별하는 도중 T 의 끝에 도달하더라도 다시 T 의 처음으로 돌아가서 알고리즘을 계속 진행한다. 문제의 답은 ‘새로운 알고리즘이 끝날 때까지 문자열 T 를 몇 바퀴 돌았는가’로 쓸 수 있다. 또한, 만약 S 의 어떤 글자에서 T 를 한 바퀴 돌았음에도 답을 얻지 못했다면, -1을 출력해야 한다. 이 때의 시간 복잡도는 $O(NM)$ 이다.

부분문제 4 ($M \leq 1\,000$)

더 빠른 시간에 작동하는 알고리즘을 만들려면, 이제 ‘ T 를 한 칸씩 도는 시간’을 절약해야 한다. T 에서의 각 인덱스 i 와 각 알파벳 문자 c 에 대해, 다음과 같은 값을 기록(전처리)한 표(2차원 배열)을 만든다.

`next[i][c]`: T 에서 i 번째 또는 그 이후에 첫번째로 등장하는 문자 c 의 인덱스

위와 마찬가지로, $\text{next}[i][c]$ 를 구할 때 T 의 처음과 끝을 이어붙여 생각하자. 이와 같은 표를 한 번 만들고 나면, 앞선 알고리즘에서 ‘ T 의 다음 인덱스를 찾는 작업’을 한 칸씩 이동하면서 하지 않아도 된다. 가야 할 ‘다음 인덱스’의 번호는 이미 우리가 만들어둔 표에 적혀 있기 때문이다. 그래서, 이제는 표를 만드는 시간을 제외하고 문제를 $O(N)$ 에 풀 수 있게 된다.

next 배열은 가장 쉬운 방법으로 만들면 $O(M^2)$ 에 만들 수 있고, 이 때에 부분문제 4를 $O(N + M^2)$ 의 시간 복잡도로 해결할 수 있게 된다.

부분문제 5

next 배열을 더 빠르게 채우면 부분문제 5를 풀 수 있다. 아이디어는 다음과 같다.

알파벳 c 를 고정하고, $\text{next}[1][c]$, $\text{next}[2][c]$, ..., $\text{next}[N][c]$ 를 모두 채우는 방법을 생각한다. 먼저, T 에서 알파벳 c 가 등장하는 인덱스들을 모두 기록해두고, 정렬된 배열로 만들어두자. 그러면 $\text{next}[i][c]$ 는 방금 만든 배열에서 i 이상인 최소 원소가 된다. 이 사실을 알면, 이분 탐색(lower_bound 등의 함수를 이용할 수 있다)을 이용해 $\text{next}[1][c]$, ..., $\text{next}[N][c]$ 를 $O(M \log M)$ 에 채우거나, 투 포인터 알고리즘을 이용해 $O(M)$ 에 채울 수 있다.

위 방법을 이용해 next 배열을 채우면 전체 문제를 $O(N + \Omega M \log M)$ 또는 $O(N + \Omega M)$ 의 시간에 풀 수 있으며, ($\Omega \leq 26$ 는 알파벳의 개수) 두 복잡도 모두 만점을 받기 충분하다.

고등부 2. 순서 섞기

풀이 작성자: 윤교준

배열 A 가 이미 비내림차순으로 정렬되어 있다면, 답은 0이다. 이제, A 가 비내림차순이 아니라고 가정한다.

부분문제 1 ($N \leq 8$)

길이 N 의 배열 A 에서 한 번의 순서 섞기 연산을 수행할 때, 나올 수 있는 결과의 종류의 수는 $O(2^N)$ 이다. 왜냐하면, 배열 A 의 좌측 혹은 우측 끝을 고르는 작업을 총 N 번 시행하기 때문이다.

‘순서 섞기’ 연산은 배열의 값의 순서만을 바꿀 뿐, 값을 바꾸지는 않는다. 고로, 배열 A 는 여러 번의 순서 섞기 연산을 통해, $O(N!)$ 종류의 값을 가질 수 있다.

따라서, BFS를 이용하여, 배열 A 가 정렬되기 위하여 필요한 연산의 최소 횟수를 $O(N! \times 2^N)$ 혹은 $O(N! \times N \times 2^N)$ 에 구할 수 있다.

부분문제 2 (답은 2 이하)

답이 0인 경우는, 배열 A 가 비내림차순인 경우이다. 이제, 답이 1인 경우의 필요충분조건을 알아보자.

답이 1이기 위한 필요조건

순서 섞기 연산을 f 라고 표현하자. 이제, 순서 섞기 연산의 역연산, 즉 f^{-1} 를 생각하자.

만일, A 에 f 를 취하여 B 를 얻었다고 하자. 이때, 배열 A 에서 ‘좌측의 수’로 선택된 수를 차례대로 배열 A_L 에 저장하고, A 에서 ‘우측의 수’로 선택된 수를 차례대로 배열 A_R 에 저장하자. A 에서, ‘좌측의 수’는 왼쪽부터 연속하게 붙어있고, ‘우측의 수’는 오른쪽부터 붙어있는 형태이다. 따라서, $A = A_L + A_R$ 임을 알 수 있다. 또한, 배열 B 는 두 배열 $A_L, \widetilde{A_R}$ 을 “원소의 상대적인 순서를 유지하며” 합친 것과 같다. 여기서, 임의의 배열 T 에 대하여, $\widetilde{T}$ 는 T 의 원소를 역순으로 배치한 배열을 의미한다.

따라서, 배열 B 에 대한 역연산 f^{-1} 는 다음과 같이 서술할 수 있다:

1. B 를 원소의 상대적인 순서를 유지하며, 두 개의 배열 B_L, B_R 로 나눈다.
2. $A = B_L + \widetilde{B_R}$

이제, 배열 A 가 단 한 번의 f 연산으로 정렬 가능하다고 가정하자. 이는, A 를 정렬한 배열 S 에 대하여, S 에 f^{-1} 연산을 취하여 A 를 얻을 수 있음을 의미한다. S 를 나눈 두 배열 S_L, S_R 은 모두 정렬되어 있다. $A = S_L + \widetilde{S_R}$ 이므로, A 는 증가하다가 감소하는 형태를 가져야 한다.

따라서, 답이 1이기 위한 필요조건은 “배열 A 가 증가하다가 감소하는 형태를 가지는 것”이다.

답이 1이기 위한 충분조건

이제, 배열 A 가 증가하다가 감소하는 형태를 가진다고 가정하자. 우리는 A 를 단 한 번의 순서 섞기 연산을 이용하여 정렬할 수 있음을 보일 것이다.

A 의 최댓값을 기준으로, (자신 포함) 왼쪽의 수를 모아둔 배열을 A_L , 오른쪽의 수를 모아둔 배열을 A_R 라고 하자. 가정에 의하여, A_L 과 $\widetilde{A_R}$ 은 정렬된 상태이다.

A 에 연산 f 를 취한 결과 배열 B 는, A_L 과 $\widetilde{A_R}$ 을 순서를 유지하며 합친 형태이다. 여기서, 합병 정렬 알고리즘의 아이디어를 이용하면, 정렬된 상태의 B 를 얻도록, 두 배열을 합치는 방법이 항상 존재함을 알 수 있다.

따라서, “증가하다가 감소하는 형태”를 가지는 배열은 항상 한 번의 연산으로 정렬할 수 있다.

위의 두 논의를 통하여, 답이 1인 경우의 필요충분조건은 “배열의 원소가 증가하다가 감소하는 형태를 가진다”임을 알 수 있다. 배열 A 가 이 조건을 만족하는지는 $O(N)$ 에 쉽게 확인할 수 있다.
 답이 2인지는, 이 부분문제에 한하여, “답이 0과 1이 아니다”라는 조건으로 확인할 수 있다.

부분문제 3 ($1 \leq A_i \leq 2$)

배열 A 에서 연속한 1들과 연속한 2들을 각각 하나의 1과 2로 합쳐도 무방하다.

배열 A 에 K 개의 2가 존재한다고 하자. 배열 A 에서 $\left\lfloor \frac{K}{2} \right\rfloor$ 번째 2를 기준으로, 이 수까지를 A_L , 이 수의 오른쪽을 A_R 로 나누자. 두 배열 $A_L, \widetilde{A_R}$ 를 합치는 적당한 방법이 존재하여, 배열 B 에 $\left\lceil \frac{K}{2} \right\rceil$ 개의 연속한 2가 존재하도록 할 수 있다.

즉, K 개의 ‘연속한 2’를, 한 번의 연산을 통해 $\left\lceil \frac{K}{2} \right\rceil$ 개의 ‘연속한 2’가 존재하도록 만들 수 있고, 이것이 최적의 전략이다.

따라서, 답은 $\lceil \log_2 K \rceil + 1$ 이다. 이에 대한 엄밀한 증명은 생략한다.

부분문제 4 (모든 A_i 가 서로 다르다)

부분문제 3의 풀이의 아이디어를 확장할 수 있다. 배열 A 가 총 K 번 ‘증가하다가 감소하는 형태’를 가진다면, 비슷한 논리로, 한 번의 연산을 이용하여, $\left\lceil \frac{K}{2} \right\rceil$ 번 ‘증가하다가 감소하는 형태’를 가지는 결과 배열을 얻을 수 있다.

이 전략 또한 최적이며, 답은 $\lceil \log_2 K \rceil + 1$ 이다.

부분문제 5

배열 A 에서 인접한 수가 서로 같다면, 이 두 수를 하나로 합쳐주어도 답이 변하지 않음을 알 수 있다. 이러한 성질을 이용하면, 부분문제 5 또한 부분문제 4와 동일한 풀이로 해결할 수 있다.

전체 시간 복잡도는 $O(N)$, 공간 복잡도는 $O(N)$ 혹은 $O(1)$ 에 해결할 수 있다.

중등부 4, 고등부 3. 화려한 정사각형

풀이 작성자: 구재현

좌표 범위의 최댓값을 X 라 표기한다.

부분문제 1 ($N, X \leq 50$)

격자 위의 정사각형은 왼쪽 아래 모서리의 x 좌표, y 좌표, 그리고 변의 길이로 표현할 수 있다. 고로 모든 가능한 $O(X^3)$ 개의 정사각형을 나열한 후, 이들 안에 들어있는 점들의 서로 다른 색이 K 개 인지를 배열에 마킹하면 된다. 전체 문제가 $O(X^3N)$ 에 해결된다.

부분문제 2 ($K \leq 50, X \leq 2500$)

왼쪽 아래 모서리의 x 좌표, y 좌표를 고정하자. 각각의 색에 대해서, 해당 색의 점을 포함하기 위해 필요한 변의 길이 최솟값을 계산하고, 이의 최댓값을 변의 길이로 취할 것이다. $DP[X][i][j]$ 를 (i, j) 를 왼쪽 모서리로 하는 정사각형이 색 X 의 점을 포함하기 위해 가져야 하는 최소 변의 길이라고 하자. 색 X 의 점이 (i, j) 위치에 있다면 이는 0이고, 아니면 이는 $\min(DP[X][i+1][j], DP[X][i+1][j+1], DP[X][i][j+1]) + 1$ 이라는 점화식으로 계산 가능하다. 전체 문제가 $O(KX^2)$ 에 해결된다.

부분문제 3 ($K = 2$)

두 점 $(x_1, y_1), (x_2, y_2)$ 를 포함하는 최소 크기 정사각형의 변의 넓이는 $\max(|x_1 - x_2|, |y_1 - y_2|)$ 이다. 이는 두 점의 L_∞ 거리 (체비셰프 거리) 와 동일하다. 색깔 1의 점과 색깔 2의 점을 각각 하나 골라서 위 값을 최소화해야 한다. 즉, 체비셰프 거리 상 가장 가까운 두 점 쌍을 찾는 것이다.

평면을 45도 회전하면, 위 거리는 $|x_1 - x_2| + |y_1 - y_2|$ 형태의 L_1 거리 (맨하탄 거리)로 변환된다. 일반성을 잃지 않고 $x_1 \leq x_2, y_1 \leq y_2$ 라고 하자 (4방향으로 모두 회전시키면서 해 보면 된다). 우리는 고정된 (x_1, y_1) 에 대해서 $x_1 \leq x_2, y_1 \leq y_2$ 를 만족하며 $x_2 + y_2$ 를 최소화하는 점을 찾아야 한다. 이는 x 좌표에 대해 스위핑을 하고, y 좌표에 대해서 $x + y$ 의 최솟값을 저장하는 세그먼트 트리를 관리하면 된다. 시간 복잡도는 $O(N \log N)$ 이다.

부분문제 4 ($N \leq 50$)

부분문제 4에서 6까지는 정사각형을 찾는다고 생각하지 않고, 각 변의 길이가 w, h 일 때 $\max(w, h)$ 를 최소화 하는 직사각형을 찾는다고 생각한다. 직사각형은 x 축에 수직인 두 변 $x = x_s, x = x_e$, y 축에 수직인 두 변 $y = y_s, y = y_e$, 이렇게 총 4개의 정수로 표현할 수 있다. 직사각형의 4개의 변 중 하나에 점이 닿지 않는다면, 점이 닿을 때까지 해당 변을 안쪽으로 당기면서 $\max(w, h)$ 를 유지하거나 줄일 수 있다. 이는 우리가 고려해야 할 (x_s, x_e, y_s, y_e) 쌍의 후보가 $O(N^4)$ 개라는 것을 뜻한다: x_s, x_e 는 입력으로 주어진 $O(N)$ 개의 x 좌표 중 하나고, y 축에 대해서도 동일하기 때문이다. 고로 모든 가능한 $O(N^4)$ 개의 정사각형을 나열한 후, 이들 안에 들어있는 점들의 서로 다른 색이 K 개 인지를 배열에 마킹하는 식으로 확인하면 된다. 시간 복잡도는 $O(N^5)$ 이다.

부분문제 5 ($N \leq 150$)

x_s, x_e 를 고정시키고, $x_s \leq x \leq x_e$ 를 만족하지 않는 점들을 모두 무시해 주자. 이제 y 좌표만이 중요하니, 1차원에서 이 문제를 해결하면 된다. 점을 y 좌표 순으로 정렬한 후, y_s 를 고정해 주고 y_e 를 늘려주면서, 서로 다른 색이 K 개가 되면 답을 갱신하자. 1차원 문제가 $O(N^2)$ 에 풀리니 전체 문제는 $O(N^4)$ 에 풀린다.

부분문제 6 ($N \leq 600$)

부분문제 5와 동일하게 1차원 문제를 해결한다. y_s 를 고정해 주고 y_e 를 늘려주면서, 서로 다른 색이 K 개인 것이 확인되면 처음부터 시작하는 것이 아니라 y_s 를 한 칸 늘려준 후 (가장 왼쪽 점을 제거한 후) 다시 y_e 를 서로 다른 색이 K 개일 때까지 늘려주는 것을 반복한다. 이러한 방식을 Sliding Window 혹은 Two Pointers라고 하며, 점이 한 번 구간에 들어가고 한 번 나가니 $O(N)$ 에 1차원 문제가 해결된다. 정렬을 해야 한다고 생각할 수 있겠지만, 맨 처음에 점을 y 좌표 기준으로 한번만 정렬해 주면 굳이 매번 정렬할 필요가 없다. 전체 문제는 $O(N^3)$ 에 풀린다.

부분문제 7 ($N \leq 2500$)

길이 R 이하의 화려한 정사각형이 존재하는가? 라는 결정 문제로 문제를 변형해서 해결한다. 이 문제를 해결할 수 있다면, 최소의 R 을 이분 탐색으로 찾을 수 있다. R 을 고정할 경우, x_s, x_e 로 가능한 후보는 $O(N)$ 개가 된다. 왼쪽 변에 점이 닿는 경우가 N 개이고, 오른쪽 변에 점이 닿는 경우가 N 개이니 $2N$ 개만 시도해 보면 되기 때문이다: 둘 다 닿지 않는 경우는 앞과 비슷하게 왼쪽이나 오른쪽 점에 부딪힐 때까지 구간을 움직여 주면 된다. 이제 부분문제 6처럼 1차원 문제를 $O(N)$ 에 푼 후 그 답이 R 이하인지를 $2N$ 번 확인해 주면, 결정 문제가 $O(N^2)$ 에 풀리고, 전체 문제는 $O(N^2 \log X)$ 에 풀린다.

부분문제 8

부분문제 7과 동일한 결정 문제를 Plane sweeping을 사용하여 효율적으로 해결한다. 길이 R 의 x 좌표 구간 $[x_c, x_c + R]$ 를 왼쪽에서 오른쪽으로 움직이자. x_c 가 증가하는 과정에서 N 개의 점들은 $x_c = x_i - R$ 시점에 구간에 삽입되고, $x_c = x_i + 1$ 시점에 구간에서 제거된다. 문제를 해결하기 위해서는, 점들이 추가되고 제거될 때, 모든 K 개의 색의 점을 포함하는 길이 R 의 구간이 있는지를 빠르게 판별해야 한다.

$distinct[y]$ 를 $[y - R, y]$ 구간을 덮었을 때 얻을 수 있는 서로 다른 색의 개수라고 정의하자. $distinct[y] = K$ 인 x 가 존재한다면, 답이 존재한다고 판별할 수 있다. 점 하나가 추가되고 제거될 때 $distinct[y]$ 가 어떠한 식으로 변경되는지 살펴보자.

각 색에 대해서, 해당 색의 점들의 y 좌표들을 `std::multiset` 과 같은 자료 구조로 관리하자. y 좌표가 y_j 인 점을 추가할 때, 기본적으로 이 점이 새롭게 $distinct[i]$ 를 1 증가시킬 구간은 $[y_j, y_j + R]$ 이 된다. 하지만, 이미 y_j 의 직전에 있는 점이 합집합을 덮고 있어서 더하지 않아도 되는 구간, 직후에 있는 점에 가로막혀서 더하지 않아도 되는 구간들이 존재한다. 이 구간은 y_j 의 양 옆으로 인접한 점들의 좌표만 알면 케이스 처리로 구할 수 있다. 고로, `std::multiset`의 `lower_bound` 함수를 사용하면 새롭게 기여하게 되는 구간을 $O(\log N)$ 에 계산할 수 있다. 삭제에서도 역시, $distinct[i]$ 가 1 감소하는 구간을 아주 유사한 방식으로 계산할 수 있다.

이제 문제는 구간에 대해서 특정 수를 더하고, 전체 배열에 값이 K 인 원소가 있는지를 체크하는 구간 쿼리 문제가 된다. 배열의 특정 원소의 값은 항상 K 이하이므로, 세그먼트 트리에 Lazy propagation을 사용하여 해결할 수 있다. 결정 문제가 $O(N \log X)$ 에 해결되니, 전체 문제도 $O(N \log^2 X)$ 에 해결되어 만점을 받을 수 있다.

고등부 4. 경계 로봇

풀이 작성자: 윤교준

하나의 센서는 $2r$ 의 구간을 감시할 수 있으므로, $2rN < L$ 라면, 답은 -1 이다. 또한, $2rN \geq L$ 라면, 항상 완벽한 경계가 가능하다. 이제, $2rN \geq L$ 가 성립한다고 가정하자.

부분문제 1 ($N \leq 10, L \leq 100, r \leq 10$)

다음 네 가지 관찰이 요구된다:

관찰 1-1 로봇이 내려놓은 센서는 다시 집어 옮기지 않아도 충분하다.

관찰 1-2 로봇이 아직 집지 않은 센서 위를 지나간다면, 그 센서를 무조건 가져가는 최적해가 존재한다.

관찰 1-3 센서의 위치는 정수만 고려해도 충분하다.

관찰 2-1 로봇이 K 개의 센서를 옮겼다면, $r, 3r, 5r, \dots, (2K-3)r, x$ 에 배치하는 것이 최적이다. 여기서, $(2K-3)r \leq x \leq (2K-1)r$ 이다.

모든 K 에 대하여, 가능한 모든 센서 배치를 생각하자. 관찰 2-1에 의하여, 이러한 배치는 $O(rN)$ 이다. (엄밀하게는, $O(L)$ 이다.)

센서 배치가 주어졌을 때, 로봇이 움직여야 하는 거리는 BFS를 통하여 계산할 수 있다. ‘현재 로봇의 위치’, ‘로봇이 집은 센서의 총 개수’, ‘배치가 완료된 센서의 집합’, 총 세 개의 정보로 현재 상태를 완벽하게 표현할 수 있다. 상태의 수가 총 $O(N2^NL)$ 이므로, $O(L \times N2^NL) = O(N2^NL^2)$ 의 시간복잡도로 부분문제를 해결할 수 있다.

부분문제 2 ($N \leq 30, L \leq 2000, r \leq 30$)

다음의 추가적인 관찰이 요구된다:

관찰 3 잉여의 센서를 가지고 있는 로봇은 무조건 전진하는 최적해가 존재한다.

센서의 최종 배치를 $Y_1, Y_2, \dots, Y_K$ 라 하자. 로봇의 최적 전략은 다음 DP를 이용하여 계산할 수 있다:

$DP[n] :=$ 현재, 로봇이 Y_n 에 위치하고, 어떠한 센서도 가지고 있지 않으며, 옮긴 n 개의 센서는 각각 $Y_1, Y_2, \dots, Y_n$ 에 위치시켰을 때, 이를 위하여 로봇이 움직여야 하는 최소 거리.

$DP[1], \dots, DP[K]$ 의 값을 모두 알고 있다면, 배치 Y_i 를 달성하기 위하여 필요한 최소 이동 거리는 $O(K^2)$ 혹은 $O(K)$ 에 계산할 수 있다.

DP 배열 또한 $O(K^3)$ 에 모든 값을 계산할 수 있다.

따라서, $O(LN^3)$ 의 시간 복잡도로 본 부분문제를 해결할 수 있다.

부분문제 3 ($N \leq 500$)

다음 관찰이 요구된다:

관찰 4-1 센서 배치 Y_i 가 주어졌을 때, 로봇이 x_l 에서 왼쪽으로 방향을 바꾸었다면, $X_i \leq x_l$ 를 만족하는 i 의 개수와 $Y_j \leq x_l$ 를 만족하는 j 의 개수가 같다.

관찰 4-2 센서 배치 Y_i 가 주어졌을 때, 로봇이 x_r 에서 오른쪽으로 방향을 바꾸었다면,

- $X_i < x_r$ 을 만족하는 i 의 개수와 $Y_j < x_r$ 을 만족하는 j 의 개수가 같으며,
- $X_i \leq x_r$ 을 만족하는 i 의 개수가 $Y_j \leq x_r$ 을 만족하는 j 의 개수보다 작다.

$Y_K = (2K - 1)r$ 을 만족하는 모든 배치 Y_i 에 대하여, 관찰 4에 의하여 로봇이 지그재그로 움직이는 과정을, 부분문제 2와 유사한 DP로 표현할 수 있다.

이 DP 배열은 $O(K^2)$ 에 계산할 수 있다. 답을 계산하는 과정은, 로봇이 마지막 센서를 처리하기 위하여 움직여야 하는 경로를 예외적으로 계산해준다면, $O(K)$ 혹은 $O(K^2)$ 에 할 수 있다.

따라서, $O(N \times N^2) = O(N^3)$ 의 시간 복잡도로 본 부분문제를 해결할 수 있다.

부분문제 4 ($N \leq 2500$)

부분문제 3의 DP를 구현하였다면, 누적 합 아이디어를 이용하여, DP 계산을 $O(K)$ 에 할 수 있다. 이 경우, 시간 복잡도는 $O(N^2)$ 이다.

부분문제 5

다음 관찰이 요구된다:

관찰 2-2 관찰 2-1에서 유의미한 x 의 값은 오직 두 개이다.

관찰 2-3 관찰 2-1에서 유의미한 K 는 오직 하나다.

따라서, 전체 시간 복잡도는 $O(N)$, 공간 복잡도는 $O(N)$ 혹은 $O(1)$ 이다.
